МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 3

«Эмпирический анализ временной сложности алгоритмов сортировки. Часть 2: алгоритмы на основе парадигмы «разделяй и властвуй» (быстрая сортировка и сортировка слиянием)»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 19

Выполнила: студентка группы БИ 2-1

Ситникова Ольга

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: изучить рекурсивные алгоритмы сортировки, построенные по принципу «разделяй и властвуй», экспериментально подтвердить их сложность O(n log n) и провести сравнительный анализ с квадратичными алгоритмами из лабораторной работы № 2.

Задание (вариант 19). Реализовать быструю сортировку (опорный элемент — **последний**, схема разбиения **Ломуто**) и **восходящую** (итеративную) сортировку слиянием; провести эксперименты 1–3; выполнить дополнительное задание Г1.

№	Опорный	Разбиение	Слияние	Размеры n для эксперимента 2	Доп. задание
19	К (последний)	Л (Ломуто)	В (восходящее)	10 000; 25 000; 40 000; 55 000; 70 000	Г1

Г1: реализовать гибридную быструю сортировку (для подмассивов длиной ≤ M применять сортировку вставками) и подобрать оптимальное $M \in \{5, 10, 16, 32, 64\}$ экспериментально.

Эксперименты. 1 — четыре алгоритма (пузырьком, вставками, быстрая, слиянием) на размерах варианта 19 ЛР № 2 (n = 1000…6000; в ЛР № 2 для этого варианта использовались почти упорядоченные данные N, поэтому таблицы приведены и для них, и для случайных данных); 2 — быстрая, слиянием, гибридная быстрая и `sorted()` на больших n; 3 — быстрая и слиянием на данных типов R, S, V, N, D.

Важное ограничение эксперимента 3. Так как в варианте опорным выбран *последний* элемент, на упорядоченных данных быстрая сортировка работает за $\Theta(n^2)$. Поэтому, согласно заданию, типы S и V исследуются при $n \le 3000$ (а для наглядности роста времени — при n = 500…4000); типы R, N, D — при n = 50 000; все пять типов дополнительно сравниваются при n = 3000.

2. Описание алгоритмов, рекуррентные соотношения и оценки сложности

Парадигма «разделяй и властвуй»: задача разбивается на подзадачи, они решаются рекурсивно, результаты комбинируются. Трудоёмкость описывается рекуррентностью $T(n) = a \cdot T(n/b) + f(n)$. При a = b = 2 и $f(n) = \Theta(n)$ по основной теореме $T(n) = \Theta(n \log n)$: дерево рекурсии имеет $\approx \log_2 n$ уровней, на каждом уровне выполняется $\Theta(n)$ операций. Любая сортировка сравнением требует $\Omega(n \log n)$ сравнений в худшем случае (дерево решений имеет не менее n! листьев).

Сортировка слиянием (восходящая). Вместо рекурсивного деления массив сразу рассматривается как n блоков длины 1; на каждом проходе соседние блоки длины w сливаются в блоки длины 2w (w = 1, 2, 4, 8, …). Проходов $\lceil \log_2 n \rceil$, каждый стоит $\Theta(n)$:

$$T(n) = \Theta(n) \cdot \lceil \log_2 n \rceil = \Theta(n \log n) \text{ — во всех случаях.}$$

Нестрогое сравнение `src[i] <= src[j]` делает сортировку устойчивой; требуется $\Theta(n)$ дополнительной памяти (буфер).

Быстрая сортировка. Опорным служит последний элемент подмассива `a[hi]`; разбиение по схеме Ломуто делит подмассив на части «≤ опорного», сам опорный элемент и «> опорного». Затем части сортируются рекурсивно. Рекурсия выполняется для *меньшей* части, а большая обрабатывается в цикле, что ограничивает глубину стека величиной O(log n). Лучший и средний случаи — $\Theta(n \log n)$ ($T(n) = 2T(n/2) + \Theta(n)$). Худший случай — $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$: для опорного «последний» он наступает на упорядоченных и обратно упорядоченных данных, так как последний

элемент оказывается наибольшим (наименьшим) и разбиение делит подмассив на части $n - 1$ и 0. Схема Ломуто с условием «≤» дополнительно плохо работает на массивах с большим числом повторов (данные типа D).

Гибридная быстрая сортировка (Г1). Рекурсия быстрой сортировки прекращается, когда длина подмассива не больше M; такие короткие подмассивы упорядочиваются вставками, поскольку при малых n константа сортировки вставками меньше накладных расходов рекурсии.

Таблица 1. Теоретические оценки

Алгоритм	Лучший	Средний	Худший	Доп. память	Устойчивость
Пузырьком	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Слиянием	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Да
Быстрая	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	Нет

Методика измерений та же, что в ЛР № 2: `time.perf_counter()`, одинаковые данные для всех алгоритмов (фиксированный seed, значения из диапазона [0; 50 000]), $k = 5$ повторов и медиана (для подбора M в Г1 — $k = 7$), проверка результата по `sorted()` вне замера, функции не изменяют исходный массив.

4. Блок-схемы алгоритмов

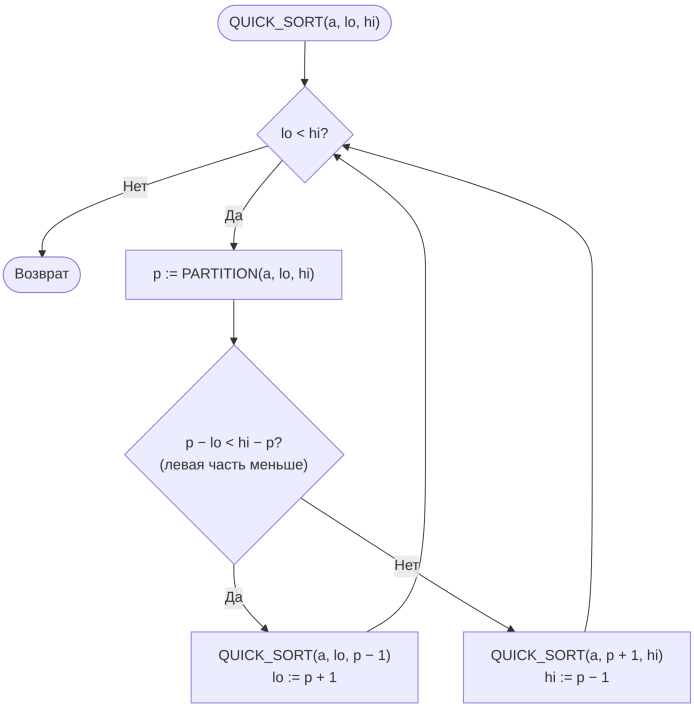


Рис. 1. Блок-схема быстрой сортировки

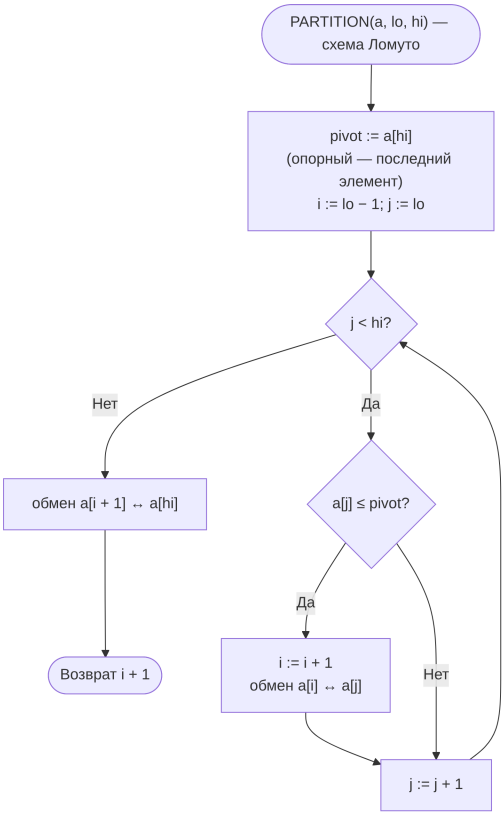


Рис. 2. Блок-схема разбиения по схеме Ломуто (опорный — последний элемент)

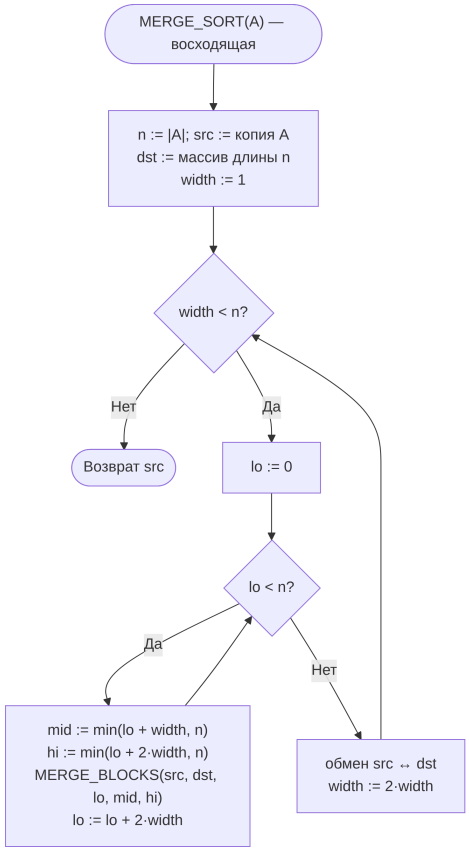


Рис. 3. Блок-схема восходящей сортировки слиянием

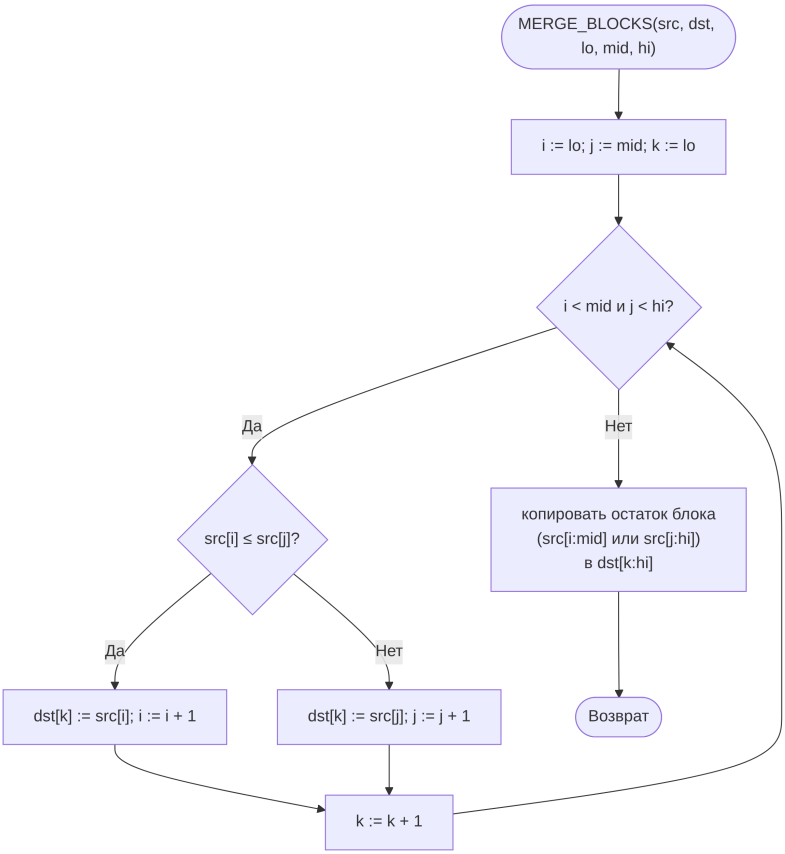


Рис. 4. Блок-схема слияния двух соседних блоков

5. Листинг программы

Файл `lr3_variant19.py` (самодостаточный модуль: функции ЛР № 2, новые алгоритмы, эксперименты и графики).

```
"""
Лабораторная работа № 3. Вариант 19.
Дисциплина: Теория алгоритмов.
Выполнила: Ситникова Ольга, группа БИ 2-1.

Алгоритмы «разделяй и властвуй»: быстрая сортировка и сортировка слиянием.

Исходные данные варианта 19:
    опорный элемент - последний (K);
    схема разбиения - Ломуто (Л);
    сортировка слиянием - восходящая, итеративная (B);
    размеры для эксперимента 2: 10 000; 25 000; 40 000; 55 000; 70 000;
    дополнительное задание Г1 - гибридная быстрая сортировка:
    для подмассивов длиной <= M применяется сортировка вставками,
    оптимальное M выбирается из {5, 10, 16, 32, 64} экспериментально.

Эксперименты:
    1 - четыре алгоритма (пузырьком, вставками, быстрая, слиянием) на
        размерах из варианта 19 ЛР № 2 (n = 1000..6000; тип данных N);
    2 - быстрая, слиянием, гибридная быстрая и sorted() на больших n;
    3 - быстрая и слиянием на данных R, S, V, N, D. Так как опорный
        элемент - последний, на упорядоченных данных быстрая сортировка
        работает за θ(n^2), поэтому типы S и V исследуются при n <= 3000;
        типы R, N, D - при n = 50 000, а все пять типов - при n = 3000.

Файл самодостаточен (функции ЛР № 2 включены сюда же).
Запуск:  python lr3_variant19.py          (график сохраняется в lr3_plot.png)
         python lr3_variant19.py --show   (дополнительно показать график)
Полный прогон занимает около полутора минут.
Требуется: matplotlib.
"""
import json
import math
import platform
import random
import statistics
import sys
import time

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # без окна: график только сохраняется в файл
import matplotlib.pyplot as plt

# ---------------------------- параметры варианта ----------------------------
SIZES_1 = [1000, 2000, 3000, 4000, 5000, 6000]    # размеры ЛР № 2 (вариант 19)
SIZES_2 = [10_000, 25_000, 40_000, 55_000, 70_000]  # эксперимент 2 (вариант 19)
SIZES_2B = [12_500, 25_000, 50_000, 100_000]      # удвоение n для T(2n)/T(n)
N_3_BIG = 50_000                                  # эксперимент 3: типы R, N, D
N_3_SMALL = 3000                                  # эксперимент 3: все типы R, S, V, N, D
SIZES_3B = [500, 1000, 2000, 4000]                # рост времени (S, V): удвоение n
M_VALUES = [5, 10, 16, 32, 64]                    # Г1: кандидаты на порог M
N_G1 = 100_000                                    # размер массива для подбора M
LO, HI = 0, 50_000                                # диапазон значений
REPEATS = 5
REPEATS_G1 = 7                                    # для подбора M (разница малая)
SEED = 42


# ---------------------------- генерация данных ----------------------------
def generate_data(n, kind="random", lo=LO, hi=HI, seed=SEED):
    """Массив длины n: random (R), sorted (S), reversed (V),
    nearly_sorted (N, n/20 случайных перестановок), dup (D, значения 0..10)."""
    rng = random.Random(seed + n)
    if kind == "dup":
        return [rng.randint(0, 10) for _ in range(n)]
    data = [rng.randint(lo, hi) for _ in range(n)]
    if kind == "sorted":
        data.sort()
    elif kind == "reversed":
        data.sort(reverse=True)
    elif kind == "nearly_sorted":
        data.sort()
        for _ in range(max(1, n // 20)):
            i, j = rng.randrange(n), rng.randrange(n)
            data[i], data[j] = data[j], data[i]
    return data
```

```
# ---------------------- алгоритмы ЛР № 2 (квадратичные) ----------------------
def bubble_sort(arr):
    """Сортировка пузырьком с флагом досрочного выхода."""
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a


def insertion_sort(arr):
    """Сортировка вставками."""
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a


# ----------------------------- быстрая сортировка -----------------------------
def quick_sort(arr):
    """Быстрая сортировка: опорный - последний элемент, разбиение Ломуто."""
    a = arr.copy()
    _quick_sort(a, 0, len(a) - 1)
    return a


def _quick_sort(a, lo, hi):
    while lo < hi:
        p = _partition_lomuto(a, lo, hi)
        # рекурсия в меньшую часть - глубина стека O(log n)
        if p - lo < hi - p:
            _quick_sort(a, lo, p - 1)
            lo = p + 1
        else:
            _quick_sort(a, p + 1, hi)
            hi = p - 1


def _partition_lomuto(a, lo, hi):
    """Разбиение Ломуто; опорный элемент - последний, a[hi]."""
    pivot = a[hi]
    i = lo - 1
    for j in range(lo, hi):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[hi] = a[hi], a[i + 1]
    return i + 1                    # итоговая позиция опорного элемента


# ---------------------- Г1: гибридная быстрая сортировка ----------------------
def hybrid_quick_sort(arr, m=16):
    """Быстрая сортировка; подмассивы длиной <= m сортируются вставками."""
    a = arr.copy()
    _hybrid(a, 0, len(a) - 1, m)
    return a


def _hybrid(a, lo, hi, m):
    while hi - lo + 1 > m:
        p = _partition_lomuto(a, lo, hi)
        if p - lo < hi - p:
            _hybrid(a, lo, p - 1, m)
            lo = p + 1
        else:
            _hybrid(a, p + 1, hi, m)
            hi = p - 1
    _insertion_range(a, lo, hi)     # короткий подмассив - вставками


def _insertion_range(a, lo, hi):
    """Сортировка вставками подмассива a[lo..hi] на месте."""
    for i in range(lo + 1, hi + 1):
        key = a[i]
```

```
        j = i - 1
        while j >= lo and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key


# --------------------- сортировка слиянием (восходящая) -----------------------
def merge_sort(arr):
    """Восходящая (итеративная) сортировка слиянием: блоки 1, 2, 4, 8, ..."""
    n = len(arr)
    src = arr.copy()
    dst = [0] * n
    width = 1
    while width < n:
        for lo in range(0, n, 2 * width):
            mid = min(lo + width, n)
            hi = min(lo + 2 * width, n)
            i, j, k = lo, mid, lo
            while i < mid and j < hi:
                if src[i] <= src[j]:         # «<=» обеспечивает устойчивость
                    dst[k] = src[i]
                    i += 1
                else:
                    dst[k] = src[j]
                    j += 1
                k += 1
            if i < mid:                      # остаток левого блока
                dst[k:hi] = src[i:mid]
            elif j < hi:                     # остаток правого блока
                dst[k:hi] = src[j:hi]
        src, dst = dst, src                  # результат слияния становится входом
        width *= 2
    return src


# ------------------------------ замер времени --------------------------------
def measure(sort_func, data, repeats=REPEATS):
    """Медианное время работы sort_func на данных data, с (perf_counter)."""
    expected = sorted(data)
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == expected, f"{sort_func.__name__}: ошибка сортировки"
    return statistics.median(times)


def run_table(algorithms, sizes, kind="random", repeats=REPEATS, scale=1.0):
    """Время каждого алгоритма для каждого n; печатает таблицу.
    Результаты возвращаются в секундах; scale=1e3 - печать в миллисекундах."""
    res = {name: [] for name in algorithms}
    print(f"{'n':>8}" + "".join(f"{name:>14}" for name in algorithms))
    for n in sizes:
        data = generate_data(n, kind)
        row = f"{n:>8}"
        for name, func in algorithms.items():
            t = measure(func, data, repeats)
            res[name].append(t)
            row += f"{t * scale:>14.4f}"
        print(row, flush=True)
    return res


def system_info():
    """Характеристики вычислительной системы (для отчёта)."""
    cpu = platform.processor() or platform.machine()
    try:
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")


def theory_ratio(n1, n2):
    """Теоретическое T(n2)/T(n1) для Θ(n log n)."""
    return (n2 * math.log2(n2)) / (n1 * math.log2(n1))
# ------------------------------------ main ------------------------------------
def main():
```

```
out = {}
print("Лабораторная работа № 3, вариант 19")
print(system_info())
print("Опорный: последний; разбиение: Ломуто; слияние: восходящее; "
      f"k = {REPEATS}\n")

# --- Г1: подбор порога M гибридной сортировки (случайные данные)
print(f"Г1. Подбор M, случайные данные, n = {N_G1}, время, с")
data = generate_data(N_G1, "random")
g1 = {"M=1 (без вставок)": measure(quick_sort, data, REPEATS_G1)}
for m in M_VALUES:
    g1[f"M={m}"] = measure(lambda a, m=m: hybrid_quick_sort(a, m), data,
                           REPEATS_G1)
for name, t in g1.items():
    print(f"{name:>20}{t:>10.4f}")
best_m = min(M_VALUES, key=lambda m: g1[f"M={m}"])
print(f"Оптимальное M = {best_m}\n")
out["g1"] = g1
out["best_m"] = best_m

def hybrid(a):
    return hybrid_quick_sort(a, best_m)
hybrid.__name__ = "hybrid_quick_sort"

# --- Эксперимент 1: четыре алгоритма
algs1 = {"Пузырьком": bubble_sort, "Вставками": insertion_sort,
         "Быстрая": quick_sort, "Слиянием": merge_sort}
print("Эксперимент 1а. Четыре алгоритма, СЛУЧАЙНЫЕ данные, с")
res1r = run_table(algs1, SIZES_1, "random")
print("\nЭксперимент 1б. Четыре алгоритма, ПОЧТИ УПОРЯДОЧЕННЫЕ данные "
      "(тип N варианта 19 ЛР № 2), с")
res1n = run_table(algs1, SIZES_1, "nearly_sorted")
out["exp1_random"] = res1r
out["exp1_nearly"] = res1n

# --- Эксперимент 2: O(n log n) на больших размерах
algs2 = {"Быстрая": quick_sort, "Слиянием": merge_sort,
         f"Гибрид M={best_m}": hybrid, "sorted()": sorted}
print("\nЭксперимент 2. Большие массивы, СЛУЧАЙНЫЕ данные, с")
res2 = run_table(algs2, SIZES_2, "random")
out["exp2"] = res2
print("\nОтношение времён (слиянием / быстрая):")
for n, tq, tm in zip(SIZES_2, res2["Быстрая"], res2["Слиянием"]):
    print(f"{n:>8}{tm / tq:>8.2f}")
print("\nОтношения T(n2)/T(n1) для соседних размеров эксперимента 2")
print(f"{'Переход':>20}{'Быстрая':>10}{'Слиянием':>10}"
      f"{'Teop.nlogn':>12}{'Teop.n^2':>10}")
ratios2 = []
for i in range(1, len(SIZES_2)):
    n1, n2 = SIZES_2[i - 1], SIZES_2[i]
    rq = res2["Быстрая"][i] / res2["Быстрая"][i - 1]
    rm = res2["Слиянием"][i] / res2["Слиянием"][i - 1]
    th, th2 = theory_ratio(n1, n2), (n2 / n1) ** 2
    ratios2.append([n1, n2, rq, rm, th, th2])
    print(f"{str(n1) + '->' + str(n2):>20}{rq:>10.2f}{rm:>10.2f}"
          f"{th:>12.2f}{th2:>10.2f}")
out["ratios2"] = ratios2

print("\nЭксперимент 2б. Удвоение n, СЛУЧАЙНЫЕ данные, с")
res2b = run_table({"Быстрая": quick_sort, "Слиянием": merge_sort},
                  SIZES_2B, "random")
print("\nT(2n)/T(n) при удвоении")
print(f"{'Переход':>20}{'Быстрая':>10}{'Слиянием':>10}{'Teop.nlogn':>12}")
ratios2b = []
for i in range(1, len(SIZES_2B)):
    n1, n2 = SIZES_2B[i - 1], SIZES_2B[i]
    rq = res2b["Быстрая"][i] / res2b["Быстрая"][i - 1]
    rm = res2b["Слиянием"][i] / res2b["Слиянием"][i - 1]
    ratios2b.append([n1, n2, rq, rm, theory_ratio(n1, n2)])
    print(f"{str(n1) + '->' + str(n2):>20}{rq:>10.2f}{rm:>10.2f}"
          f"{theory_ratio(n1, n2):>12.2f}")
out["exp2b"] = res2b
out["ratios2b"] = ratios2b

# --- Эксперимент 3: структура данных
kinds = [("random", "R"), ("sorted", "S"), ("reversed", "V"),
         ("nearly_sorted", "N"), ("dup", "D")]
print(f"\nЭксперимент 3а. Типы R, N, D при n = {N_3_BIG}, с "
      "(S и V при таком n недопустимы: опорный - последний)")
print(f"{'Тип':>6}{'Быстрая':>12}{'Слиянием':>12}{'Быстрая/Слиянием':>20}")
exp3_big = {}
for kind, code in [("random", "R"), ("nearly_sorted", "N"), ("dup", "D")]:
    data = generate_data(N_3_BIG, kind)
    tq = measure(quick_sort, data)
    tm = measure(merge_sort, data)
```

```
    exp3_big[code] = [tq, tm]
    print(f"{code:>6}{tq:>12.4f}{tm:>12.4f}{tq / tm:>20.2f}", flush=True)
out["exp3_big"] = exp3_big

print(f"\nЭксперимент 3б. Влияние структуры данных, n = {N_3_SMALL}, с")
print(f"{'Тип':>6}{'Быстрая':>12}{'Слиянием':>12}{'Быстрая/Слиянием':>20}")
exp3 = {}
for kind, code in kinds:
    data = generate_data(N_3_SMALL, kind)
    tq = measure(quick_sort, data)
    tm = measure(merge_sort, data)
    exp3[code] = [tq, tm]
    print(f"{code:>6}{tq:>12.4f}{tm:>12.4f}{tq / tm:>20.2f}", flush=True)
out["exp3_small"] = exp3

print("\nЭксперимент 3в. Рост времени на УПОРЯДОЧЕННЫХ данных (S), с")
res3s = run_table({"Быстрая": quick_sort, "Слиянием": merge_sort},
                  SIZES_3B, "sorted")
print("\nЭксперимент 3г. Рост времени на ОБРАТНО УПОРЯДОЧЕННЫХ данных (V), с")
res3v = run_table({"Быстрая": quick_sort, "Слиянием": merge_sort},
                  SIZES_3B, "reversed")
out["exp3_sorted_growth"] = res3s
out["exp3_reversed_growth"] = res3v
print("\nT(2n)/T(n) на упорядоченных данных (теория: n^2 -> 4, n log n -> ~2,1)")
for i in range(1, len(SIZES_3B)):
    n1, n2 = SIZES_3B[i - 1], SIZES_3B[i]
    print(f"{str(n1) + '->' + str(n2):>14}"
          f"{res3s['Быстрая'][i] / res3s['Быстрая'][i - 1]:>8.2f}"
          f"{res3s['Слиянием'][i] / res3s['Слиянием'][i - 1]:>8.2f}")

# --- Сводная таблица (n = 5000, случайные данные)
i5 = SIZES_1.index(5000)
t_bubble = res1r["Пузырьком"][i5]
print("\nСводная таблица, n = 5000, случайные данные")
print(f"{'Алгоритм':>12}{'Время, с':>12}{'Ускорение':>12}")
for name in algs1:
    t = res1r[name][i5]
    print(f"{name:>12}{t:>12.4f}{t_bubble / t:>12.1f}")

# --- Графики
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
for name, t in res1r.items():
    axes[0, 0].plot(SIZES_1, t, marker="o", label=name)
axes[0, 0].set_yscale("log")
axes[0, 0].set_title("Эксперимент 1 (случайные данные, лог. шкала)")
for name, t in res2.items():
    axes[0, 1].plot(SIZES_2, t, marker="o", label=name)
axes[0, 1].set_title("Эксперимент 2 (большие массивы)")
for ax in axes[0]:
    ax.set_xlabel("Размер массива n")
    ax.set_ylabel("Время, с")
    ax.grid(True)
    ax.legend()
codes = [c for _, c in kinds]
xs = range(len(codes))
axes[1, 0].bar([x - 0.2 for x in xs], [exp3[c][0] for c in codes], 0.4,
               label="Быстрая")
axes[1, 0].bar([x + 0.2 for x in xs], [exp3[c][1] for c in codes], 0.4,
               label="Слиянием")
axes[1, 0].set_xticks(list(xs))
axes[1, 0].set_xticklabels(codes)
axes[1, 0].set_yscale("log")
axes[1, 0].set_title(f"Эксперимент 3 (n = {N_3_SMALL}, лог. шкала)")
axes[1, 0].set_xlabel("Тип данных")
axes[1, 0].set_ylabel("Время, с")
axes[1, 0].legend()
axes[1, 0].grid(True, axis="y")
for name, t in res3s.items():
    axes[1, 1].plot(SIZES_3B, t, marker="o", label=name)
axes[1, 1].set_title("Упорядоченные данные (S): рост времени")
axes[1, 1].set_xlabel("Размер массива n")
axes[1, 1].set_ylabel("Время, с")
axes[1, 1].grid(True)
axes[1, 1].legend()
fig.tight_layout()
fig.savefig("lr3_plot.png", dpi=150)

fig2, ax2 = plt.subplots(figsize=(6, 4))
ax2.plot([1] + M_VALUES, list(g1.values()), marker="o")
ax2.set_xscale("log")
ax2.set_title(f"Г1: подбор M (n = {N_G1})")
ax2.set_xlabel("Порог M (M = 1 — без вставок)")
ax2.set_ylabel("Время, с")
ax2.grid(True)
fig2.tight_layout()
```

```
    fig2.savefig("lr3_plot_g1.png", dpi=150)
    print("\nГрафики сохранены: lr3_plot.png, lr3_plot_g1.png")

    if "--json" in sys.argv:                    # служебный вывод для отчёта
        with open(sys.argv[sys.argv.index("--json") + 1], "w", encoding="utf-8") as f:
            json.dump(out, f, ensure_ascii=False, indent=1)
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
    main()
```

6. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.30GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; соотношения между алгоритмами и отношения $T(n_2)/T(n_1)$ должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

7. Результаты экспериментов

Таблица 2. Эксперимент 1а — четыре алгоритма, случайные данные, с

n	Пузырьком	Вставками	Быстрая	Слиянием
1 000	0,0242	0,0114	0,0006	0,0007
2 000	0,1039	0,0483	0,0012	0,0016
3 000	0,2342	0,1079	0,0018	0,0026
4 000	0,4169	0,1896	0,0031	0,0035
5 000	0,6449	0,2988	0,0036	0,0047
6 000	0,9503	0,4375	0,0045	0,0059

Таблица 3. Эксперимент 1б — четыре алгоритма, почти упорядоченные данные (тип N варианта 19 ЛР № 2), с

n	Пузырьком	Вставками	Быстрая	Слиянием
1 000	0,0151	0,0016	0,0027	0,0006
2 000	0,0637	0,0089	0,0027	0,0015
3 000	0,1495	0,0132	0,0043	0,0023
4 000	0,2456	0,0226	0,0067	0,0031
5 000	0,4087	0,0381	0,0109	0,0044
6 000	0,5883	0,0524	0,0080	0,0054

Таблица 4. Эксперимент 2 — алгоритмы O(n log n) на больших массивах (случайные данные), с

n	Быстрая	Слиянием	Гибрид (M = 16)	`sorted()`	Слиянием / быстрая
10 000	0,0086	0,0107	0,0082	0,0010	1,25
25 000	0,0227	0,0304	0,0214	0,0028	1,34
40 000	0,0399	0,0499	0,0369	0,0048	1,25
55 000	0,0519	0,0704	0,0485	0,0070	1,36
70 000	0,0703	0,0959	0,0648	0,0091	1,36

Таблица 5. Проверка роста: $T(n_2)/T(n_1)$ для соседних размеров эксперимента 2

Переход	Быстрая	Слиянием	Теория O(n log n)	Теория O(n²)
10 000 → 25 000	2,64	2,84	≈ 2,75	6,25
25 000 → 40 000	1,76	1,64	≈ 1,67	2,56
40 000 → 55 000	1,30	1,41	≈ 1,42	1,89
55 000 → 70 000	1,35	1,36	≈ 1,30	1,62

Таблица 6. Эксперимент 2б — удвоение n (случайные данные): время, с, и T(2n)/T(n)

n	Быстрая, с	Слиянием, с	Переход	Быстрая	Слиянием	Теория
12 500	0,0107	0,0137	—	—	—	—
25 000	0,0223	0,0292	12 500 → 25 000	2,09	2,14	≈ 2,15
50 000	0,0520	0,0636	25 000 → 50 000	2,33	2,18	≈ 2,14
100 000	0,1039	0,1385	50 000 → 100 000	2,00	2,18	≈ 2,13

Таблица 7. Эксперимент 3а — типы R, N, D при n = 50 000, с (S и V при таком n недопустимы)

Тип данных	Быстрая	Слиянием	Быстрая / слиянием
Случайные (R)	0,0524	0,0643	0,81
Почти упорядоченные (N)	0,1114	0,0599	1,86
С повторами (D)	5,2721	0,0543	97,10

Таблица 8. Эксперимент 3б — влияние структуры данных при n = 3000, с

Тип данных	Быстрая	Слиянием	Быстрая / слиянием
Случайные (R)	0,0019	0,0026	0,74
Упорядоченные (S)	0,2074	0,0017	124,18
Обратно упорядоченные (V)	0,1441	0,0016	89,10
Почти упорядоченные (N)	0,0062	0,0034	1,82
С повторами (D)	0,0274	0,0024	11,24

Таблица 9. Эксперимент 3в — рост времени на упорядоченных (S) и обратно упорядоченных (V) данных, с, и T(2n)/T(n)

n	S: быстрая	S: слиянием	V: быстрая	V: слиянием
500	0,0048	0,0002	0,0038	0,0002
1 000	0,0218	0,0005	0,0158	0,0005
2 000	0,0910	0,0010	0,0639	0,0010
4 000	0,3729	0,0023	0,2543	0,0024
T(2n)/T(n), среднее	4,26	2,23	4,07	2,26

Таблица 10 (Г1). Подбор порога M гибридной сортировки (n = 100 000, случайные данные, k = 7), с

M	1 (без вставок)	5	10	16	32	64
Время, с	0,1059	0,1020	0,0960	0,0959	0,1066	0,1216

Минимум достигнут при M = 16: 0,0959 с против 0,1059 с без вставок (выигрыш ≈ 9 %).

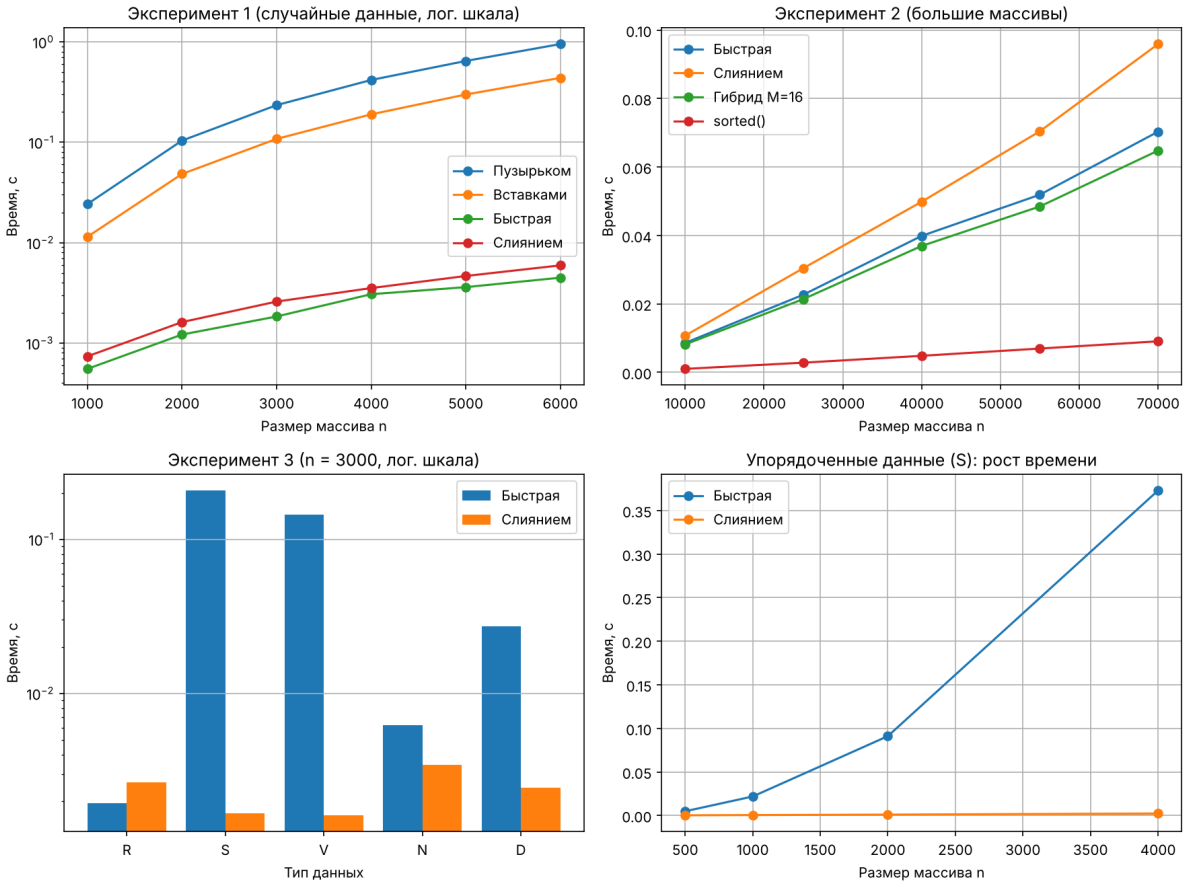


Рис. 5. Графики экспериментов: 1 (лог. шкала), 2, 3 (лог. шкала, n = 3000) и рост времени на упорядоченных данных

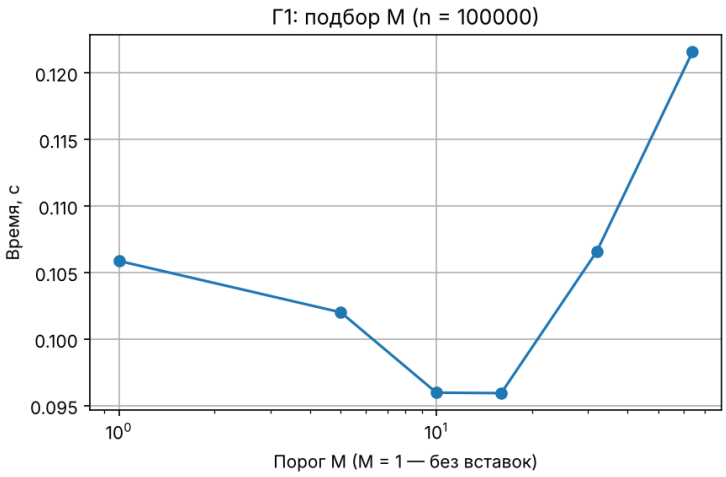


Рис. 6. Г1: зависимость времени гибридной сортировки от порога M

8. Сводная сравнительная таблица

Таблица 11. Итоговое сравнение алгоритмов (n = 5000, случайные данные; рост — T(4000)/T(2000), теория для $\Theta(n^2)$ — 4, для $\Theta(n \log n)$ ≈ 2,18)

Алгоритм	Теор. сложность (средняя)	Время, с	Ускорение относительно пузырька	Эксп. рост T(4000)/T(2000)	Память	Устойчивость
Пузырьком	$\Theta(n^2)$	0,6449	1	4,01	O(1)	Да
Вставками	$\Theta(n^2)$	0,2988	2,2	3,92	O(1)	Да
Слиянием	$\Theta(n \log n)$	0,0047	138	2,18	$\Theta(n)$	Да
Быстрая	$\Theta(n \log n)$	0,0036	179	2,52	O(log n)	Нет

9. Выводы и рекомендации по выбору алгоритма

1. **Классы сложности различаются принципиально.** На случайных данных при n = 5000 быстрая сортировка быстрее пузырька в 179 раз, сортировка слиянием — в 138 раз; относительно сортировки вставками — в 83 и 64 раз. На логарифмической шкале (рис. 5) кривые квадратичных алгоритмов растут значительно круче, разрыв увеличивается с ростом n.
2. **Почти упорядоченные данные (тип N, табл. 3).** Здесь выигрывает адаптивность: сортировка вставками при n = 6000 занимает 0,0524 с против 0,4375 с на случайных данных (в 8 раз быстрее), пузырёк — 0,588 с. Тем не менее слияние (0,0054 с) и быстрая сортировка (0,0080 с) остаются быстрее вставок примерно в 7–10 раз; при этом быстрая сортировка с последним опорным на таких данных заметно медленнее, чем на случайных (0,0080 с против 0,0045 с).
3. **Подтверждение оценки Θ(n log n).** При удвоении n на случайных данных время быстрой сортировки и слияния возрастает в 2,00–2,33 раза (табл. 6), теория даёт ≈ 2,13–2,15, квадратичный рост дал бы 4. Для размеров варианта 19 (табл. 5) измеренные отношения $T(n_2)/T(n_1)$ близки к теоретическим для n log n (например, 10 000 → 25 000: 2,64 и 2,84 при теории 2,75) и существенно ниже квадратичных (6,25). Отклонения вызваны кэш-памятью и накладными расходами интерпретатора.
4. **На случайных данных быстрая сортировка быстрее слияния** в 1,25–1,36 раза (табл. 4): она работает на месте, а слияние на каждом проходе перекладывает все элементы в буфер. Разрыв меньше, чем у рекурсивной версии слияния из образца задания (1,6–1,7 раза); вероятная причина — восходящая версия использует один заранее выделенный буфер и не создаёт срезов списков.
5. **Опорный «последний» элемент и структура данных (эксперимент 3).** На упорядоченных (S) и обратно упорядоченных (V) данных последний элемент всегда оказывается наибольшим или наименьшим, разбиения получаются максимально несбалансированными (n − 1 и 0), и $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. При n = 3000 быстрая сортировка тратит 0,2074 с (S) и 0,1441 с (V) против 0,0019 с на случайных данных, то есть в 106 и 74 раз медленнее, и в 124 и 89 раз медленнее слияния. При удвоении n время растёт в 4,1–4,5 раза (теория для n^2 — 4), тогда как у слияния — примерно в 2,1–2,4 раза (табл. 9). Поэтому по условию задания эти данные исследованы при n ≤ 4000: оценка для n = 50 000 по закону n^2 даёт ≈ 58 с на одну сортировку (оценка, не измерение) против ≈ 0,06 с у слияния. Без приёма «рекурсия только в меньшую часть» глубина стека достигала бы n и приводила к `RecursionError` (лимит ≈ 1000 вызовов), здесь же глубина остаётся малой.
6. **Другие типы данных.** На почти упорядоченных данных (N, n = 50 000) быстрая сортировка в 2,1 раза медленнее, чем на случайных (0,1114 с против 0,0524 с), но

вырождения нет: случайные перестановки нарушают порядок. На данных с повторами (D, значения 0…10) схема Ломуто с условием «≤» вырождается: 5,27 с, то есть в 101 раз медленнее, чем на случайных, и в 97 раз медленнее слияния (0,0543 с): все элементы, равные опорному, попадают в одну часть. Сортировка слиянием от структуры данных практически не зависит.

7. **Гибридизация (Г1).** Оптимальный порог M = 16 дал выигрыш около 9 % на n = 100 000 (табл. 10); значения M = 5…16 различаются в пределах погрешности измерения, а M = 64 уже хуже чистой быстрой сортировки (0,1216 с против 0,1059 с). В эксперименте 2 гибридная версия быстрее обычной на 5–8 %. Гибридизация не устраняет вырождение на упорядоченных данных: оно связано с выбором опорного элемента.
8. **Встроенная сортировка.** Функция `sorted()` быстрее собственной быстрой сортировки в 7,5–8,5 раза, а собственной сортировки слиянием — в 10,1–10,8 раза (реализация на C, гибридный адаптивный алгоритм). В прикладных информационных системах следует использовать стандартные библиотечные средства сортировки.
9. **Рекомендации по выбору.** Данные в информационных системах часто уже упорядочены (журналы, выгрузки по дате), поэтому первый или последний элемент в качестве опорного выбирать не следует: нужны средний, случайный элемент или медиана трёх, а при множестве повторов — трёхпутевое разбиение. Если нужны устойчивость (многоуровневая сортировка отчётов) или гарантированное время $\Theta(n \log n)$ независимо от данных — сортировка слиянием. Если важна экономия памяти и средняя скорость — быстрая сортировка с защитой от вырождения, лучше в гибридном варианте. Малые ($n < 50$) и почти упорядоченные массивы — сортировка вставками. Для данных, не помещающихся в память, нужна внешняя сортировка на основе слияния.

10. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 2, 4, 7, 8.
2. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 2.
3. Hoare C. A. R. Quicksort // The Computer Journal. — 1962. — Vol. 5, No. 1. — P. 10–16.
4. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.
5. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.